



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

DE52 Software Engineering

11 mars 2025

Introduction to Software Design and Architecture



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Introduction

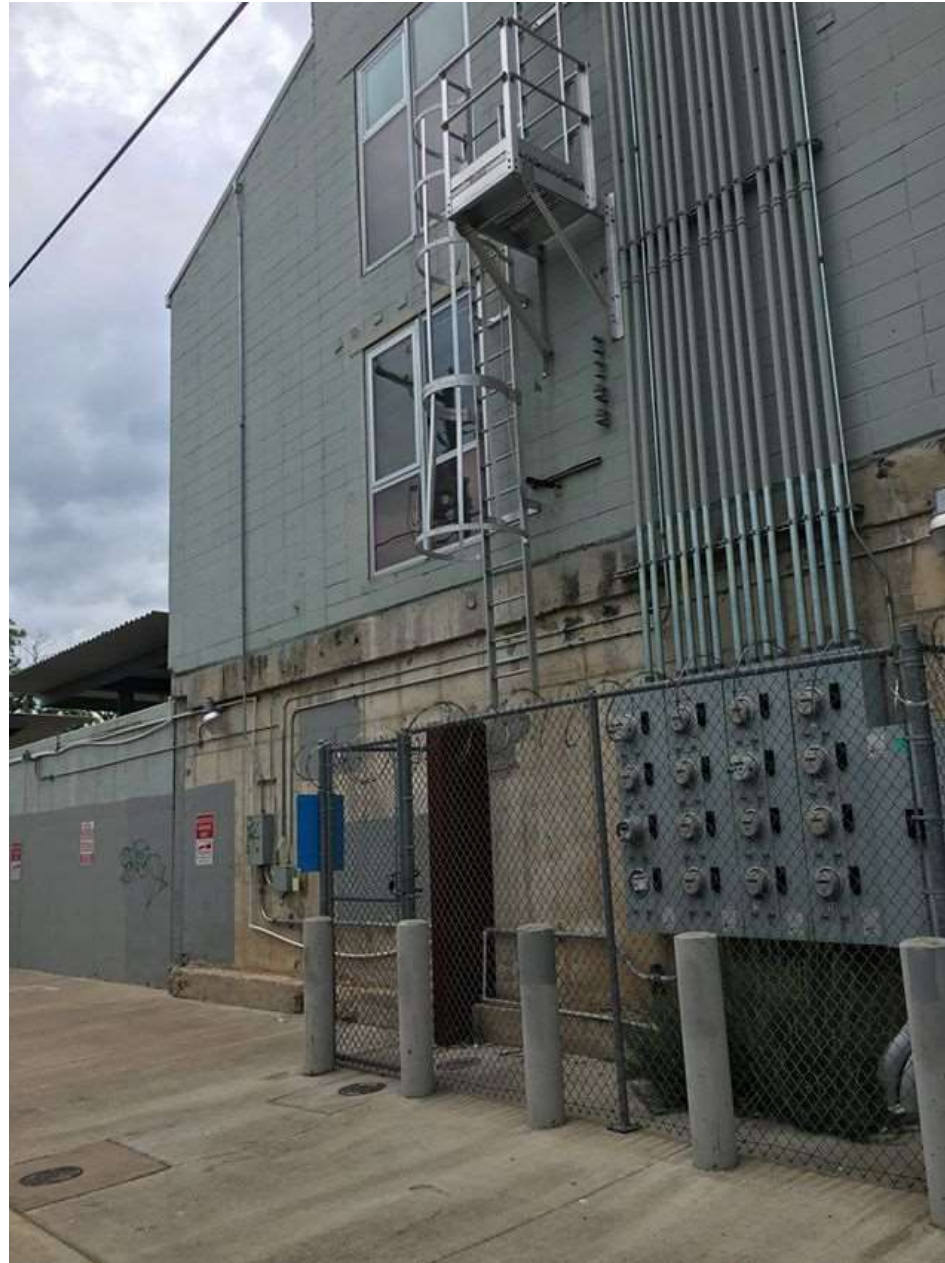
Examples of bad design



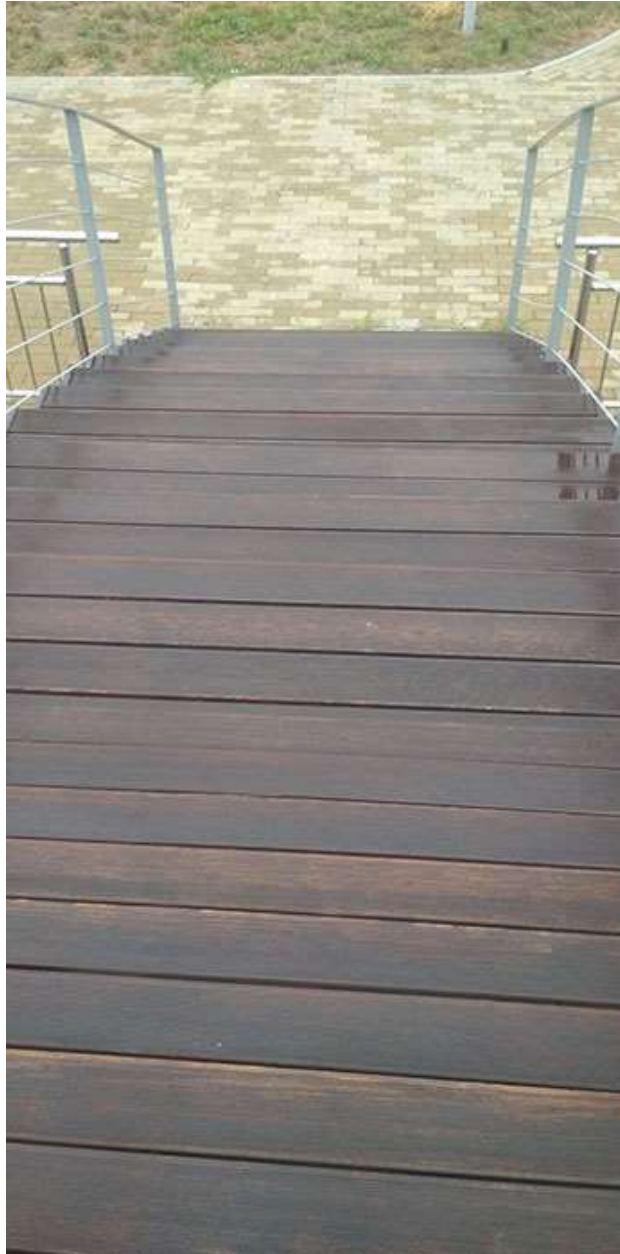
Examples of bad design



Examples of bad design



Examples of bad design



Examples of bad design



Examples of bad design

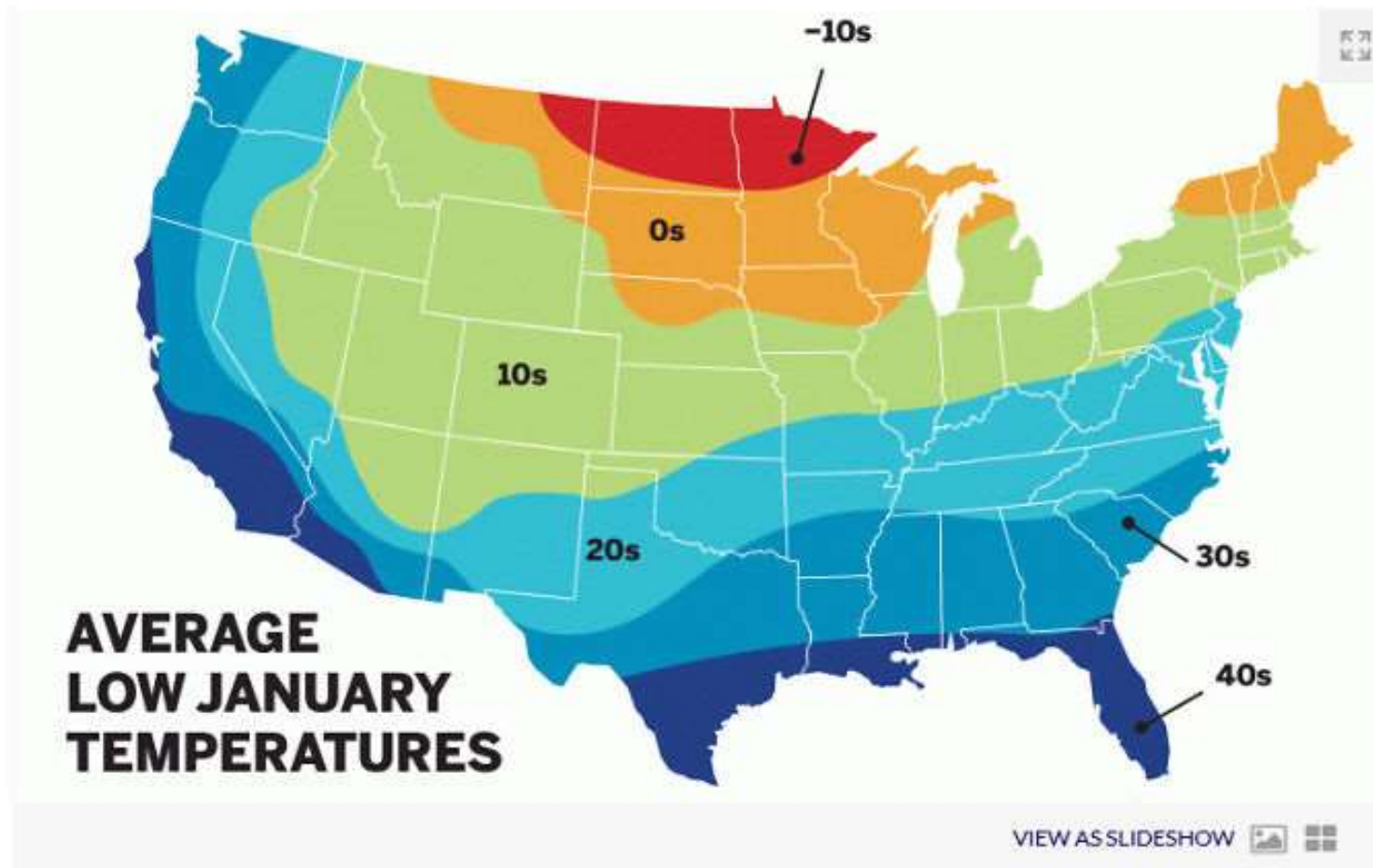


Examples of bad design



Examples of bad design

Heat Pumps Move North



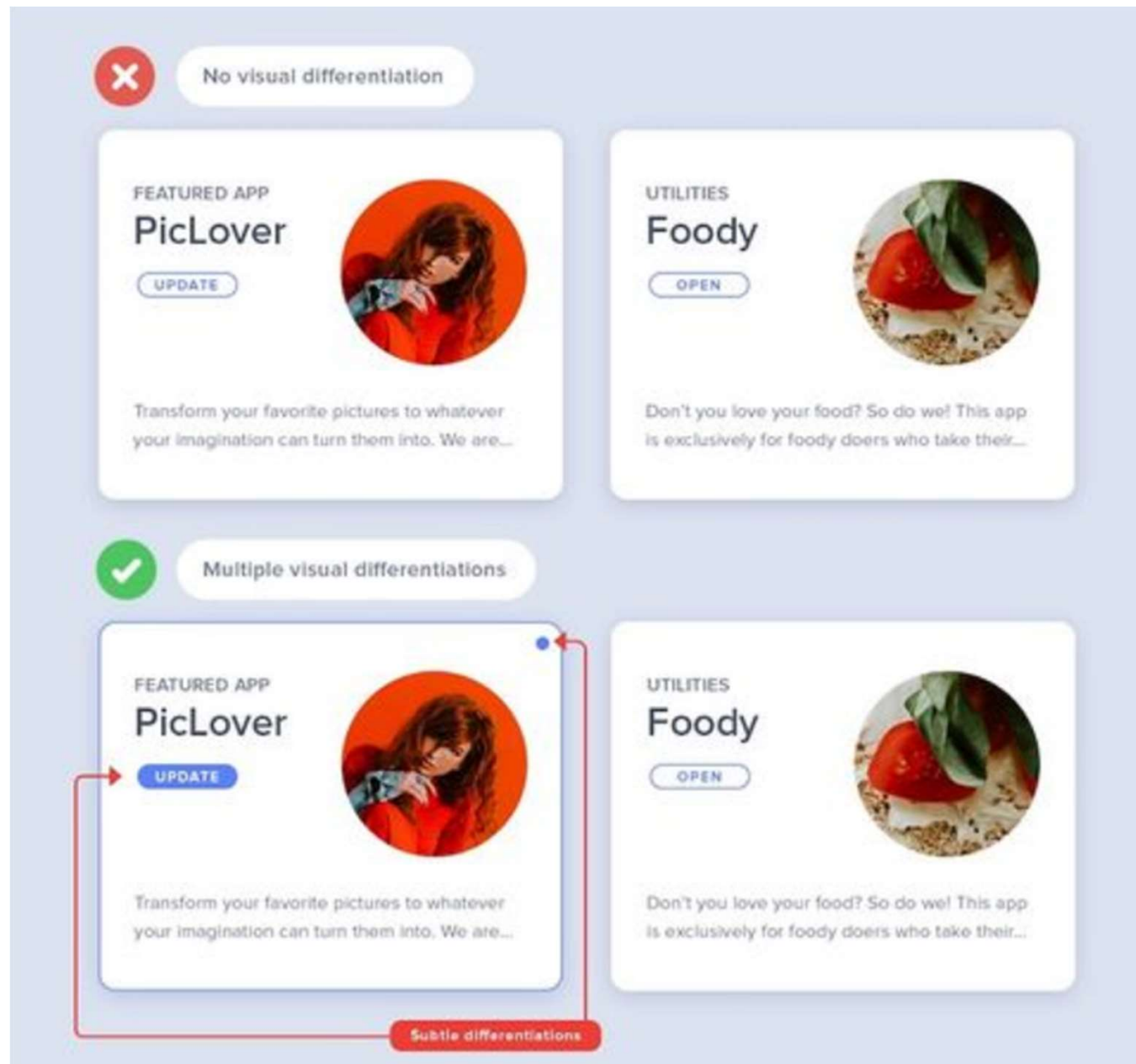
Examples of bad design



Examples of bad design



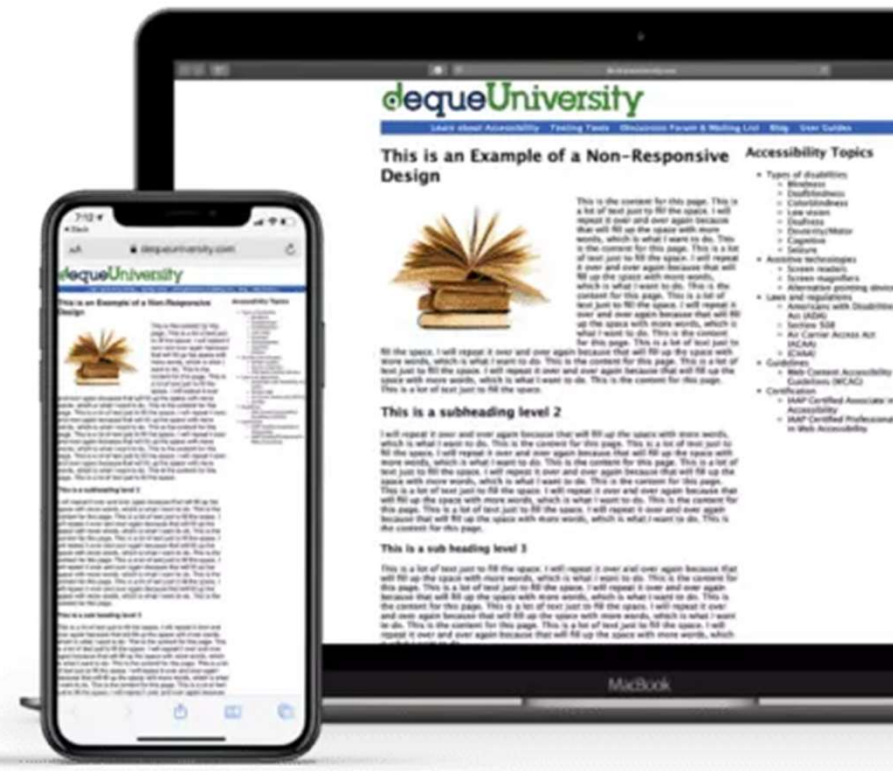
Examples of bad design on the web



Examples of bad design on the web



Examples of bad design on the web



Examples of bad design on the web



Critical thinking about design and architecture

- Questions to reflect about regarding software
 - Did they have a good design?
 - Could the design be done better?
 - Was there even a design at all?
 - How do you know if the software was well-designed?
 - Did a small code change produce a ripple-effect for changes elsewhere in the code?
 - Was your code hard to reuse?
 - Was the software difficult to maintain after a release?

The bad design

- Scenario 1: joining a project that's been in development for a while
 - You look at the code and become instantly overwhelmed
 - You can't tell what the purpose of the pieces are
 - Things are unorganized
 - Design documentation is non-existent
 - You don't even know where to begin

- Scenario 2: Working on a personal development project
 - You weren't quite sure what the functionalities would be, but you just started coding
 - It didn't matter that the code was unorganized because you were the only one working on it and you know how it works
 - Changes break the program elsewhere

The good design

- Express ideas for your software with other developers, other teams, and your clients
- Having a well-thought design makes your software easier to implement, reduces a need for major changes later on the project
- Help your software become **flexible, reusable, and maintainable**

DE52: Planning and goals

- Goals:
 - Apply design principles, patterns, and architectures to create reusable and flexible software applications and systems
 - Express in document the design and architecture of a software system using a visual notation - Unified Modeling Language (UML)
 - Handle practical examples and opportunities for you to apply this knowledge (in the TDs)

- ACM/IEEE Software Engineering Curriculum Guidelines

Modules

- Software engineering
 - Life Cycle Models
 - Agile approaches for software development (SCRUM approach)
 - Requirement's analysis and Specifications
- Object-oriented analysis and design:
 - Create flexible, reusable, and maintainable software by applying object-oriented design principles
 - Unified Modeling Language (UML)
 - Based on java
- Design patterns:
 - Design issues, problems and applications
 - Gain a foundation for more complex software applications
 - Identify problematic software designs

Modules

■ Software architectures:

- Organizing the software systems found in industry today
- Specific UML diagrams will express important architectural perspectives.
- Analyze and evaluate a given architecture by examining the trade-offs
- Discuss service-oriented architecture and learn some architectures for web applications



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Software Architect and Design Roles in Industry

Software Architect and Designer Roles

■ Questions:

- What does a career in software design and architecture look like?
- What is the difference between software design and software architecture?

■ Software designer vs. Software architect:

- Company size, the scope of the project, the experience of the development team, the organizational structure and the age of the company
- In some companies, there may be a distinct role for a software designer or architect. In other companies, the design may be completed by a member or members of the development team

Software Architect and Designer Roles

- Software designer role: outlining a software solution to a specific problem by designing the details of individual components and their responsibilities
- Software architect role: looking at the entire system and choosing appropriate frameworks, data storage, solutions and determining how components interact with each other
- In short:
 - Software design looks at the lower-level aspects of a system, whereas software architecture tends to look at the bigger picture, the higher-level aspects of a system
 - Think of this like designing a building. An architect focuses on the major structures and services, while an interior designer focuses on the smaller spaces within
 - Both are detail-oriented, forward thinkers

Software Architecture and Design Concepts

- Why is software design and architecture important?
 - Design is important if you want to have a stable, long- lived system
 - Architecture is important because if you get it wrong, your project will fail

- What are key challenged in software design and architecture?
 - One of the key challenges is the tendency to have to **trade off** between features, e.g. speed and quality
 - One of the biggest issues to face is understanding the client's problem

Software Architecture and Design Concepts

- What does a software architect do?
 - A software architect's job is to be the interface between the product and the customer and the engineering teams
 - The software architect is like a building architect. They're responsible for the overall conceptual integrity of the project

- What skills are needed in being a software designer or architect?
 - Deep technical expertise and meta skills of technologies
 - Empathic communication and soft people skills
 - Functional skills like a little bit of project planning and organizational skills
 - Meta skills about various technologies and ideas

Software Architecture and Design Concepts

■ How to improve yourself?

- Exposing yourself to as much as you can in the industry
- Read! A lot of code, general technical articles, and academic journals
- Talk to / work with people who are better than you are
- Learn from your mistakes

References

- UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design
- Design Patterns: Elements of Reusable Object-Oriented Software
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- Software Design and Architecture by Kenny Wong, University of Alberta Computing Science, Faculty of Science, Coursera
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- “Object-Oriented Patterns,” P. Coad; Comm. of the ACM, 9/92
- “Documenting Frameworks using Patterns,” R. Johnson; OOPSLA '92

Images Sources and Acknowledgments

- <https://wandr.studio/blog/bad-website-design/>
- <https://www.sitebuilderreport.com/blog/examples-of-bad-design-in-the-real-world>
- <https://www.demilked.com/hilarious-examples-of-bad-design/>
- <https://www.reddit.com/r/CrappyDesign/comments/cth6kl/thats how i broke my leg/>
- <https://www.reddit.com/r/CrappyDesign/comments/dfghom/i hope the electrician knows the sprinklers/>
- <https://www.reddit.com/r/mildlyinfuriating/comments/d9ojso/our company now has 900 of these pens/>
- <https://www.reddit.com/r/CrappyDesign/comments/ddyxgs/this backroad near my house />
- <https://www.reddit.com/r/mildlyinfuriating/comments/9fanxa/these two pens at my office nothing can possibly/>
- <https://www.reddit.com/r/CrappyDesign/comments/d963x3/this elevator/>
- <https://www.reddit.com/r/CrappyDesign/comments/d1t87g/lets use red to indicate the coldest temperatures/>



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

End of Lecture